

Lakebase + Unity Catalog + Databricks Apps

Guia técnico e didático — arquitetura, integração e validação

Baseado no capstone Customer 360 (Acme Retail), construído e validado ao vivo no workspace `e2-demo-field-eng`.

Sumário

1. O que é o Lakebase e por que ele existe
 2. A integração Lakebase ↔ Unity Catalog (reverse ETL / synced tables)
 3. Identidade e permissões: quem pode ler o quê
 4. Cenário de teste ponta a ponta (validável)
 5. Como um app usa o Lakebase (padrão de conexão)
 6. A camada analítica: SQL Warehouse, Dashboard AI/BI e Genie
 7. Forward ETL: devolvendo dados do Lakebase para o gold
 8. Checklist de validação
-

1. O que é o Lakebase e por que ele existe

O **Lakebase** é um **PostgreSQL gerenciado** dentro do Databricks. Ele existe para resolver um problema que o Delta Lake / SQL Warehouse não resolvem bem: **leituras transacionais de baixíssima latência** (sub-10ms) e **escritas OLTP** — o tipo de acesso que um aplicativo web faz a cada clique.

Necessidade	Ferramenta certa
Analytics, agregações, joins pesados sobre milhões de linhas	Delta gold + SQL Warehouse
Ler o perfil de 1 cliente em < 10ms para renderizar uma tela	Lakebase (Postgres)
Gravar uma nota / um override que o usuário digitou	Lakebase (Postgres)

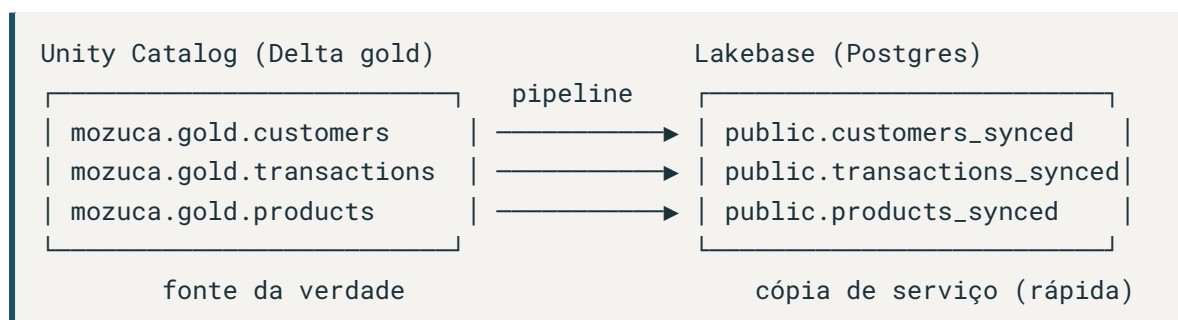
A ideia central do padrão: o dado "fonte da verdade" analítico vive no Delta gold (Unity Catalog), e o Lakebase serve como camada de serviço rápida para o app — mantida em sincronia automática com o gold.

Dois tiers de Lakebase. *Autoscaling* (comandos `databricks postgres`, escala a zero, branching nativo) e *Provisioned* (comandos `databricks database`, capacidade fixa `CU_1/2/4/8`). O capstone usa **Provisioned** (`capstone-pg`, `CU_1`, banco `capstone_db`). Os conceitos abaixo valem para ambos; os comandos mudam.

2. A integração Lakebase ↔ Unity Catalog

2.1 O conceito: synced tables (reverse ETL)

Uma **synced table** é uma tabela Postgres no Lakebase que é **alimentada automaticamente a partir de uma tabela Delta do Unity Catalog**. Você não copia dados na mão — o Databricks roda um pipeline gerenciado que mantém a cópia fresca.



2.2 Os três modos de sincronização

Ao criar a synced table você escolhe uma **scheduling policy** — e essa escolha é uma decisão de arquitetura, não um detalhe:

Modo	Como funciona	Quando usar
CONTINUOUS	Pipeline sempre ligado; mudanças no gold aparecem no Postgres em segundos	Dados que o app precisa refletir ao vivo (ex.: perfil do cliente, transações)
TRIGGERED	Sincroniza sob demanda / em agenda (ex.: de hora em hora)	Dados lentos (ex.: catálogo de 200 produtos) — evita manter pipeline ligado à toa

Modo	Como funciona	Quando usar
SNAPSHOT	Uma cópia única, sem atualização	Carga inicial ou dados imutáveis

Decisão real no capstone:

- `customers_synced` e `transactions_synced` → **CONTINUOUS** (o rep precisa ver o dado atual).
- `products_synced` → **TRIGGERED** (catálogo quase estático; pipeline contínuo seria desperdício).

2.3 Criando uma synced table (Provisioned tier)

O nome da tabela no UC (`capstone_lakebase.public.customers_synced`) mapeia para uma tabela Postgres real (`public.customers_synced`) dentro do banco `capstone_db` .

```
{
  "name": "capstone_lakebase.public.customers_synced",
  "database_instance_name": "capstone-pg",
  "logical_database_name": "capstone_db",
  "spec": {
    "source_table_full_name": "mozuca.gold.customers",
    "primary_key_columns": ["customer_id"],
    "scheduling_policy": "CONTINUOUS",
    "create_database_objects_if_missing": true,
    "new_pipeline_spec": {
      "storage_catalog": "mozuca",
      "storage_schema": "gold"
    }
  }
}
```

```
databricks database create-synced-database-table \
  --json @customers_synced.json --profile e2-demo-field-eng
```

Verificando o estado (deve chegar a **ONLINE**):

```
databricks database get-synced-database-table \
  capstone_lakebase.public.customers_synced --profile e2-demo-field-eng \
  | jq -r '.data_synchronization_status.detailed_state'
# CONTINUOUS: SYNCED_TABLE_ONLINE_CONTINUOUS_UPDATE
# TRIGGERED: SYNCED_TABLE_ONLINE_NO_PENDING_UPDATE
```

2.4 Duas classes de tabela no Lakebase

Além das synced tables (que são **somente-leitura** para o app — quem escreve nelas é

o pipeline), você cria **staging tables graváveis** com DDL Postgres normal. É onde o app grava o que o usuário produz:

```
-- Notas que o usuário deixa. processed=false até o forward-ETL levar ao gold.
CREATE TABLE customer_notes_staging (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id TEXT          NOT NULL,
  note        TEXT          NOT NULL,
  author_email TEXT        NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  processed   BOOLEAN      NOT NULL DEFAULT false
);

-- Log de auditoria append-only (uma linha por escrita do app).
CREATE TABLE customer_audit_log (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  actor_email TEXT          NOT NULL,
  action       TEXT          NOT NULL,
  customer_id TEXT          NOT NULL,
  detail       JSONB,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Resumo mental:

- **synced tables** = espelho rápido do gold (o app **lê**).
- **staging tables** = caixa de entrada do app (o app **escreve**); depois o forward-ETL leva ao gold.

3. Identidade e permissões: quem pode ler o quê

Este é o ponto que mais gera erro na prática. Há **duas identidades** em jogo:

Identidade	O que é	Usada para
Service Principal (SP) do app	A identidade "de sistema" do app	Todo acesso ao Lakebase e disparo de jobs
OBO (On-Behalf-Of)	A identidade do usuário logado , repassada pelo proxy do Apps	SQL Warehouse e Genie (para RLS e auditoria refletirem o usuário real)

3.1 Por que o Lakebase usa SEMPRE o SP

O Lakebase **não suporta escopos OBO** hoje. Se você tentar gerar credencial Postgres com o bearer OBO do usuário, recebe:

```
Provided OAuth token does not have required scopes: postgres
```

Por isso: **todo acesso ao banco roda como o SP do app**, e o e-mail do usuário chamador é lido do header `X-Forwarded-Email` só para gravar na auditoria.

3.2 O role do SP precisa existir e ter grants

Um SP fresco **não tem privilégio nenhum** no Postgres. São dois passos:

```
-- 1) Criar o role do SP no Lakebase (o nome do role é o client_id do SP)
SELECT databricks_create_role('2148e212-5a16-4e92-b056-dcaca07c956d', 'SERVICE_PRINCIPAL')

-- 2) Conceder privilégios
GRANT USAGE ON SCHEMA public TO "2148e212-...";
GRANT SELECT ON customers_synced, transactions_synced, products_synced TO "2148e212-...";
GRANT SELECT, INSERT, UPDATE ON customer_notes_staging, customer_segment_overrides_staging TO "2148e212-...";
GRANT SELECT, INSERT ON customer_audit_log TO "2148e212-...";
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO "2148e212-...";
-- Para futuras synced tables (re-sync recria) herdarem acesso:
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO "2148e212-...";
```

3.3 Para a camada analítica (warehouse), grants no Unity Catalog

Quando o app consulta o **gold via SQL Warehouse** (seja como SP ou OBO), a identidade

precisa de grants no **Unity Catalog** — não no Postgres:

```
databricks grants update catalog mozuca \
  --json '{"changes":[{"principal":"<sp_ou_usuario>","add":["USE_CATALOG"]}]}'
```

```
databricks grants update schema mozuca.gold \
  --json '{"changes":[{"principal":"<sp_ou_usuario>","add":["USE_SCHEMA","SELECT"]}]}'
```

Erro clássico: User does not have USE CATALOG on Catalog 'mozuca' → falta o grant de UC acima. Não confunda com os grants Postgres da seção 3.2.

4. Cenário de teste ponta a ponta (validável)

Este roteiro prova que a integração funciona **de verdade**. Cada passo tem uma verificação objetiva.

Teste A — A synced table reflete o gold

Objetivo: confirmar que os dados do Delta gold chegaram ao Postgres.

```
P=e2-demo-field-eng
HOST=$(databricks database get-database-instance capstone-pg --profile $P -o json | jq -r '.host')
TOKEN=$(databricks database generate-database-credential --json '{"instance_names":["capstone-pg"]' -o json | jq -r '.token')
EMAIL=$(databricks current-user me --profile $P -o json | jq -r '.userName')

PGPASSWORD=$TOKEN psql "host=$HOST port=5432 dbname=capstone_db user=$EMAIL sslmode=require"
SELECT 'customers' t, count(*) FROM customers_synced
UNION ALL SELECT 'transactions', count(*) FROM transactions_synced
UNION ALL SELECT 'products', count(*) FROM products_synced;"
```

✓ **Resultado esperado (dados do capstone):**

t	count
customers	10000
transactions	100600
products	200

Teste B — CONTINUOUS realmente propaga mudanças

Objetivo: provar que uma alteração no gold aparece no Postgres em segundos.

1. Atualize uma linha no gold via SQL Warehouse:

```
sql UPDATE mozuca.gold.customers SET city = 'TESTE-SYNC' WHERE customer_id = 'C0000400';
```

2. Aguarde ~10s e leia no Postgres:

```
sql SELECT customer_id, city FROM customers_synced WHERE customer_id = 'C0000400';
```

✓ **Resultado esperado:** `city = 'TESTE-SYNC'` (a mudança propagou sozinha).
(Reverta depois para não sujar o dado.)

Teste C — Escrita transacional na staging + auditoria

Objetivo: provar que uma escrita do app grava a nota e a auditoria na **mesma transação**, e que o override de segmento é **idempotente**.

```

BEGIN;
INSERT INTO customer_notes_staging (customer_id, note, author_email)
  VALUES ('C0000400', 'Cliente pediu retorno', 'ana@empresa.com') RETURNING id;
INSERT INTO customer_audit_log (actor_email, action, customer_id, detail)
  VALUES ('ana@empresa.com', 'add_note', 'C0000400', '{"note_id": 1}');
COMMIT;

-- Idempotência: reenviar o mesmo segmento NÃO deve duplicar nem alterar updated_at
INSERT INTO customer_segment_overrides_staging (customer_id, segment_id, author_email)
VALUES ('C0000400', 'S1', 'ana@empresa.com')
ON CONFLICT (customer_id) DO UPDATE
  SET segment_id = EXCLUDED.segment_id, updated_at = now(), processed = false
  WHERE customer_segment_overrides_staging.segment_id <> EXCLUDED.segment_id;

```



Resultado esperado:

- 1 nota + 1 linha de auditoria (contadas juntas).
- Reenviar S1 idêntico → **nenhuma** linha nova e updated_at **inalterado**.
- Trocar para S2 → updated_at muda e nova linha de auditoria.

Teste D — O role do SP tem exatamente os grants certos

```

SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = '<client_id_do_sp>'
ORDER BY table_name, privilege_type;

```



Resultado esperado: SELECT nas 3 synced; SELECT/INSERT/UPDATE nas staging; SELECT/INSERT na auditoria. **Nunca** INSERT / UPDATE nas synced (elas são mantidas pelo pipeline, não pelo app).

5. Como um app usa o Lakebase

5.1 O padrão de conexão que funciona

O token OAuth do Lakebase **expira em ~1 hora**. O padrão robusto é um **pool de conexões** que gera um token fresco a cada nova conexão e recicla conexões **antes** do token vencer:

```

import psycopg
from psycopg_pool import ConnectionPool
from databricks.sdk import WorkspaceClient

w = WorkspaceClient() # no Apps, usa as credenciais do SP automaticamente

class OAuthConnection(psycopg.Connection):
    """Injeta um token OAuth fresco como senha a cada conexão nova."""
    @classmethod
    def connect(cls, conninfo="", **kwargs):
        cred = w.database.generate_database_credential(instance_names=["capstone-pg"])
        kwargs["password"] = cred.token
        return super().connect(conninfo, **kwargs)

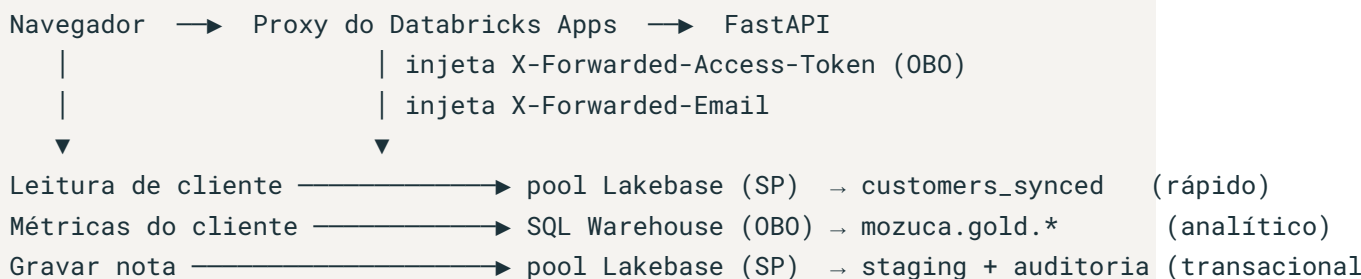
pool = ConnectionPool(
    conninfo="dbname=capstone_db user=<SP_CLIENT_ID> host=<PGHOST> port=5432 sslmode=require",
    connection_class=OAuthConnection,
    min_size=2, max_size=10,
    max_lifetime=2700, # 45 min - recicla ANTES do token de 1h expirar
    open=False, # abre no startup do app, não no import
)

```

Por que assim:

- `max_lifetime=2700` evita o bug clássico de servir uma conexão com token quase vencido (o default de 3600 cria uma corrida).
- `open=False` + abrir no `lifespan` do FastAPI faz o app **falhar rápido** se o banco estiver inacessível, em vez de aceitar requisições que travam.

5.2 Fluxo de uma requisição no app



- **Leituras de tela** → Lakebase via SP (latência baixa).
- **Métricas / agregações** → SQL Warehouse com a identidade OBO do usuário.
- **Escritas** → Lakebase via SP, sempre pareadas com a auditoria na mesma transação.

5.3 Autenticação: SP vs OBO no código

```
# SP – identidade de sistema (Lakebase, jobs)
def sp_client() -> WorkspaceClient:
    return WorkspaceClient() # no Apps, credenciais do SP são injetadas

# OBO – identidade do usuário (warehouse, Genie)
def obo_client(request) -> WorkspaceClient:
    token = request.headers["X-Forwarded-Access-Token"]
    cfg = Config(host=HOST, token=token, auth_type="pat") # força PAT
    return WorkspaceClient(config=cfg)
```

Armadilha real (vista no deploy): no runtime do Apps existem variáveis

DATABRICKS_CLIENT_ID/SECRET do SP. Se você criar o

WorkspaceClient(token=...) sem

auth_type="pat", o SDK vê "oauth + pat" e falha com *"more than one authorization*

method configured". O auth_type="pat" resolve.

Pré-requisito do OBO: o workspace precisa do toggle **Settings → Apps → User**

authorization (preview) ligado, e o app declara os escopos em app.yaml

(user_authorization: scopes: [sql, dashboards.genie]). Sem o toggle, os escopos são

aceitos mas **não ficam efetivos** — o token OBO chega sem sql /

dashboards.genie e

as chamadas ao warehouse/Genie retornam 401. Sintoma de diagnóstico:

user_api_scopes

mostra os escopos, mas effective_user_api_scopes mostra só os defaults da plataforma.

6. A camada analítica

O app tem três superfícies analíticas, todas sobre o **Delta gold** (não o Lakebase):

6.1 SQL Warehouse — agregações cruzadas ao vivo

Métricas que cruzam várias tabelas gold (gasto vitalício, top categorias, tickets, CSAT) rodam via **Statement Execution API** no warehouse, com a identidade OBO:

```

resp = client.statement_execution.execute_statement(
    warehouse_id=WAREHOUSE_ID,
    statement="""
        SELECT p.category, SUM(t.amount) AS total
        FROM mozuca.gold.transactions t
        JOIN mozuca.gold.products p ON p.product_id = t.product_id
        WHERE t.customer_id = :cid AND t.status = 'completed'
        GROUP BY p.category ORDER BY total DESC LIMIT 5
    """,
    parameters=[{"name": "cid", "value": "C0000400"}],
    wait_timeout="30s",
)

```

Por que warehouse e não Lakebase aqui? Porque é uma **agregação analítica** sobre 100k+ transações × produtos × tickets — exatamente o que o motor Delta/Photon faz bem e

o Postgres de serviço não. Regra: **transacional/pontual → Lakebase; analítico/agregado → warehouse.**

6.2 Dashboard AI/BI embedado

O app expõe `GET /api/config` com `{host, dashboard_id}` e o front renderiza um `iframe`:

```
https://<workspace-host>/embed/dashboardsv3/<dashboard_id>
```

Pré-requisito: o domínio do app precisa estar na allowlist em **Settings → Security → External Access → Embed Dashboard**, senão o `iframe` é bloqueado por `X-Frame-Options`.

6.3 Genie — perguntas em linguagem natural

O Genie responde perguntas ad-hoc ("Top 5 segmentos por LTV") via a **Conversation API**, sempre com a identidade OBO (as respostas respeitam as permissões do usuário):

```

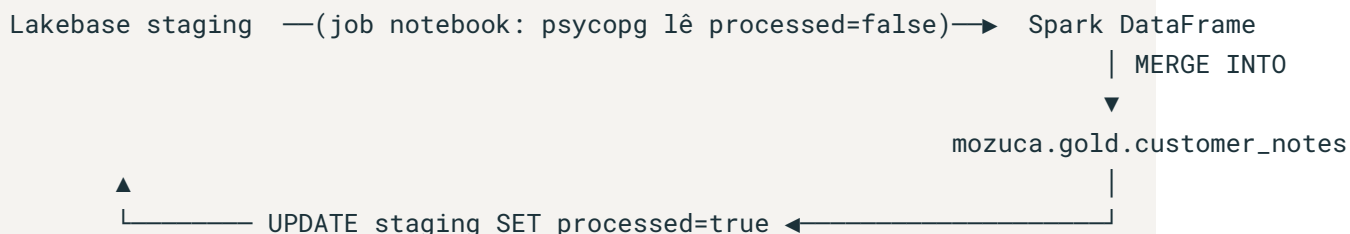
genie = obo_client(request).genie
conv = genie.start_conversation(space_id=GENIE_SPACE_ID, content="Top 5 segmentos por LTV")
# poll de get_message até status COMPLETED; se houver attachment de query, buscar o resul

```

O Genie retorna **texto + o SQL gerado + uma prévia do resultado** — o app mostra os três. Validado ao vivo: a pergunta "Top 5 segmentos por LTV total" retornou Champions \$20.3M, Loyal \$10.3M, At Risk \$7.0M, etc., com o SQL que ele mesmo gerou.

7. Forward ETL: devolvendo dados do Lakebase para o gold

As notas e overrides que o app grava na staging precisam voltar ao **Delta gold** (para analytics, ML, auditoria de longo prazo). Isso é o **forward-ETL** (Pattern A):



- Lê as linhas `WHERE processed = false`.
- `MERGE INTO` no gold (idempotente — reprocessar não duplica).
- Marca `processed = true` na mesma passada.
- Disparado pela tela **Reports** do app via Jobs API.

Idempotência garantida: rodar de novo sem linhas novas é no-op (o filtro `processed=false` não pega nada); o `MERGE` por chave evita duplicatas.

8. Checklist de validação

#	Verificação	Como confirmar
1	3 synced tables em estado ONLINE	<code>get-synced-database-table</code> → <code>*_ONLINE_*</code>
2	Contagens batem com o gold	Teste A (10000 / 100600 / 200)
3	CONTINUOUS propaga em segundos	Teste B (update no gold aparece no Postgres)
4	Escrita nota+auditoria é atômica	Teste C (1 nota ⇒ 1 auditoria)
5	Override é idempotente	Teste C (reenvio igual = no-op)
6	Role do SP tem grants corretos	Teste D (SELECT nas synced, R/W nas staging)
7	App lê Lakebase via pool	log do app: <code>lakebase pool opened</code>
8	Warehouse (SP) lê o gold	<code>/api/segments</code> → 200 (após grant UC)
9	Métricas/Genie via OBO	requer toggle <i>User authorization</i> ON + escopos efetivos

#	Verificação	Como confirmar
10	Forward-ETL é idempotente	rodar 2× sem dados novos = no-op

Princípios que ficam

1. Fonte da verdade no gold (UC); Lakebase é a camada de serviço rápida.
2. Synced tables = leitura (pipeline mantém); staging tables = escrita (app grava).
3. Lakebase sempre pelo SP; warehouse/Genie pela identidade OBO do usuário.
4. Transacional → Lakebase; analítico → SQL Warehouse.
5. Toda escrita do app carrega uma linha de auditoria na mesma transação.
6. O que entra na staging volta ao gold pelo forward-ETL, de forma idempotente.